

DAY 19: PROBABILITY DISTRIBUTIONS FOR DATA SCIENCE

Muhammad Ahmad

60-Day Data Science Learning Journey

Goal: Learn probability distributions deeply enough to use them in Statistics, Data Science and Machine Learning.

Prerequisite: Day 18 - Probability Fundamentals

1. What Is a Probability Distribution?

A probability distribution describes how probability is spread across the possible values of a random variable. It helps us understand not only one outcome, but the pattern of possible outcomes.

2. Discrete vs Continuous Distributions

Discrete: countable values such as number of students, defects, calls or Heads.
Continuous: measurements across a range such as height, weight, temperature and time.

3. PMF, PDF and CDF

PMF: Probability Mass Function, used for discrete variables. It gives the probability of a specific value, such as $P(X = 3)$.

PDF: Probability Density Function, used for continuous variables. PDF height represents density. For continuous data, probability comes from area over an interval, not the PDF height at one exact point.

CDF: Cumulative Distribution Function. It gives $P(X \leq x)$, the probability accumulated up to x .

Concept	Used for	Meaning
PMF	Discrete	Probability of a specific value
PDF	Continuous	Probability density
CDF	Discrete or Continuous	Probability $X \leq x$

4. Bernoulli Distribution

Bernoulli describes one binary trial with two outcomes. Examples include Pass/Fail, Yes/No and Heads/Tails. We commonly represent success as 1 and failure as 0.

```
from scipy.stats import bernoulli

p = 0.8
distribution = bernoulli(p)

print(distribution.pmf(1))
print(distribution.pmf(0))
```

Line by line: Import Bernoulli. Set $p=0.8$ for an 80% success probability. Create the distribution. `pmf(1)` gives 0.8 and `pmf(0)` gives 0.2.

5. Binomial Distribution

Binomial counts the number of successes in a fixed number of independent Bernoulli trials, with the same success probability on each trial.

```
from scipy.stats import binom

n = 10
p = 0.5

probability = binom.pmf(6, n, p)
print(probability)
```

Expected: 0.205078125, about 20.51%. This is the probability of exactly 6 successes in 10 trials.

Student example: if the probability of passing is 0.8 for each of 10 independent students:

```
from scipy.stats import binom

probability = binom.pmf(8, 10, 0.8)
print(probability)
```

Expected: approximately 0.301989888, or 30.20%.

6. Binomial CDF

```
from scipy.stats import binom

probability = binom.cdf(8, 10, 0.8)
print(probability)
```

cdf(8) means $P(X \leq 8)$, the probability that 8 or fewer students pass.

7. Normal Distribution

The Normal distribution is a continuous bell-shaped distribution described mainly by its mean and standard deviation. Not every real dataset is normally distributed, so this assumption should be checked rather than automatically made.

The standard Normal distribution has mean 0 and standard deviation 1.

8. Z-Score

A z-score measures how many standard deviations a value is above or below the mean: $z = (x - \text{mean}) / \text{standard deviation}$.

For $x=85$, $\text{mean}=70$ and $\text{standard deviation}=10$, $z=(85-70)/10=1.5$. So 85 is 1.5 standard deviations above the mean.

```
from scipy.stats import norm

mean = 70
std = 10

probability = norm.cdf(85, mean, std)
print(probability)
```

The result is approximately 0.9332, meaning about 93.32% under this Normal model.

9. Uniform Distribution

A uniform distribution has equal density across a specified interval.

```
from scipy.stats import uniform

probability = uniform.cdf(0.7)
print(probability)
```

For Uniform(0,1), the result is 0.7.

10. Expected Value, Variance and Standard Deviation

Expected value is the long-run average outcome. For a discrete variable, $E(X) = \sum xP(x)$. Variance measures spread around the mean. Standard deviation is the square root of variance.

Example: values [1,2,3] with probabilities [0.2,0.5,0.3] have expected value $1 \times 0.2 + 2 \times 0.5 + 3 \times 0.3 = 2.1$.

11. NumPy Simulation

```
import numpy as np

results = np.random.binomial(
    n=10,
    p=0.5,
    size=100
)

print(results[:10])
```

Each result is the number of successes in one group of 10 trials. The exact output changes because the simulation is random.

12. Student Pass Simulation

```
import numpy as np

results = np.random.binomial(
    n=10,
    p=0.8,
    size=1000
)

passed = np.sum(results)
print("Passed:", passed)
```

Each trial represents one student. 1 means pass and 0 means fail. The exact result changes between runs.

13. Practice Tasks

1. Bernoulli with $p=0.7$: calculate success and failure probabilities.
2. Coin tossed 10 times: calculate probability of exactly 5 Heads.
3. Pass probability 0.8 for 10 students: probability exactly 8 pass.
4. Same model: probability 8 or fewer pass.
5. Calculate z-score for $x=60$, $\text{mean}=50$, $\text{standard deviation}=5$.
6. Calculate $P(X \leq 60)$ using SciPy Normal with $\text{mean}=50$ and $\text{std}=5$.
7. Simulate 1,000 Binomial experiments with $n=10$ and $p=0.5$.
8. Uniform(0,100): calculate $P(X \leq 40)$.
9. For marks [55,60,65,70,72,75,78,80,85,90], calculate mean, median, standard deviation and z-score of 90.
10. Build a Student Performance Distribution Analyzer using Pandas, NumPy and SciPy.

14. Complete Solutions

```
from scipy.stats import bernoulli
```

```
d = bernoulli(0.7)
print(d.pmf(1))
print(d.pmf(0))
```

Explanation: Output: 0.7 and 0.3.

```
from scipy.stats import binom
print(binom.pmf(5, 10, 0.5))
```

Explanation: Output: approximately 0.24609375.

```
from scipy.stats import binom
print(binom.pmf(8, 10, 0.8))
```

Explanation: Output: approximately 0.301989888.

```
from scipy.stats import binom
print(binom.cdf(8, 10, 0.8))
```

Explanation: This calculates $P(X \leq 8)$.

```
x = 60
mean = 50
std = 5
z = (x - mean) / std
print(z)
```

Explanation: Output: 2.0.

```
from scipy.stats import norm
print(norm.cdf(60, 50, 5))
```

Explanation: Output: approximately 0.97725.

```
import numpy as np
results = np.random.binomial(10, 0.5, 1000)
print(results[:10])
```

Explanation: The output changes on every run.

```
from scipy.stats import uniform
print(uniform.cdf(40, loc=0, scale=100))
```

Explanation: Output: 0.4.

```
import numpy as np

marks = np.array([55, 60, 65, 70, 72, 75, 78, 80, 85, 90])
mean = marks.mean()
median = np.median(marks)
std = marks.std()
z = (90 - mean) / std

print(mean)
```

```
print(median)
print(std)
print(z)
```

Explanation: NumPy std() uses the population formula by default.

```
import pandas as pd
from scipy.stats import binom, norm

data = {
    "Name": ["Ali", "Usman", "Hamza", "Ahmad", "Bilal",
             "Hassan", "Zain", "Omar", "Ayan", "Saad"],
    "Marks": [85, 72, 91, 65, 78, 88, 95, 70, 82, 75]
}

df = pd.DataFrame(data)

mean_marks = df["Marks"].mean()
std_marks = df["Marks"].std()

df["Z_Score"] = (
    df["Marks"] - mean_marks
) / std_marks

exact_8_pass = binom.pmf(8, 10, 0.8)
eight_or_fewer = binom.cdf(8, 10, 0.8)

print("Mean:", mean_marks)
print("Std:", std_marks)
print(df)
print("P(exactly 8 pass):", exact_8_pass)
print("P(8 or fewer pass):", eight_or_fewer)
```

Explanation: This combines Pandas, z-scores and Binomial probabilities. Pandas std() uses the sample standard deviation by default.

15. Full Student Performance Distribution Analyzer

This project combines the major Day 19 concepts into one practical workflow.

```
import pandas as pd
from scipy.stats import binom, norm

data = {
    "Name": [
        "Ali", "Usman", "Hamza", "Ahmad",
        "Bilal", "Hassan", "Zain", "Omar",
        "Ayan", "Saad"
    ],
    "Marks": [85, 72, 91, 65, 78, 88, 95, 70, 82, 75]
}

df = pd.DataFrame(data)

mean_marks = df["Marks"].mean()
std_marks = df["Marks"].std()

df["Z_Score"] = (
    df["Marks"] - mean_marks
) / std_marks

above_average = df[df["Marks"] > mean_marks]

highest_z = df.loc[df["Z_Score"].idxmax()]

exact_8_pass = binom.pmf(8, 10, 0.8)
eight_or_fewer = binom.cdf(8, 10, 0.8)

normal_probability = norm.cdf(
    85, mean_marks, std_marks
)

print("Average Marks:", mean_marks)
print("Standard Deviation:", std_marks)
print("\nStudent Z-Scores:")
print(df)
print("\nAbove Average Students:")
print(above_average)
print("\nHighest Z-Score Student:")
print(highest_z)
print("\nP(exactly 8 pass):", exact_8_pass)
print("P(8 or fewer pass):", eight_or_fewer)
print("\nP(X <= 85) using Normal model:", normal_probability)
```

Line-by-line project explanation

1. Import Pandas for tabular data.
2. Import Binomial and Normal distributions from SciPy.
3. Create the student dataset.
4. Build a DataFrame.
5. Calculate the class average.
6. Calculate sample standard deviation with Pandas.
7. Convert every mark into a z-score.
8. Filter students above the average.
9. Find the highest z-score.
10. Calculate theoretical probability of exactly 8 passes.
11. Calculate probability of 8 or fewer passes.
12. Use a Normal model to estimate $P(X \leq 85)$.
13. Print the results.

16. Distribution Comparison

Distribution	Type	Main use	Example
Bernoulli	Discrete	One binary trial	Pass/Fail
Binomial	Discrete	Count successes in fixed trials	8 passes out of 10
Normal	Continuous	Bell-shaped continuous model	Measurements
Uniform	Continuous	Equal density across interval	Random 0-100

17. Common Mistakes

- Assuming every dataset is Normal.

- Confusing PMF and PDF.

- Treating $PDF(x)$ as probability at an exact continuous point.

- Forgetting Binomial independence and fixed success probability assumptions.

- Expecting simulation to exactly equal theoretical probability.

- Mixing population and sample standard deviation without checking

the definition.
• Forgetting that CDF represents cumulative probability.

18. Quick Revision

PMF: discrete probability of a value.
PDF: continuous probability density.
CDF: probability up to a value.
Bernoulli: one binary trial.
Binomial: successes in fixed independent trials.
Normal: continuous bell-shaped model.
Uniform: equal density over an interval.
Z-score: standardized distance from mean.

19. Final Checklist

■ I understand probability distributions.
■ I can distinguish discrete and continuous variables.
■ I understand PMF, PDF and CDF.
■ I can use Bernoulli and Binomial distributions.
■ I understand Normal and Uniform distributions.
■ I can calculate z-scores.
■ I can simulate distributions with NumPy.
■ I can use SciPy probability distributions.
■ I can apply distributions to a practical student dataset.

20. Day 19 Takeaway

Probability distributions connect probability theory with practical Data Science. They help us model uncertainty, standardize observations, simulate outcomes and reason about patterns in data.